

HERMÈS

Approfondissement

Orchestrateur de workflows, bundle client, accessibilité, i18n, OpenAPI et migration PostgreSQL

Ce volume conclut la série d'audit en traitant les risques résiduels non couverts par les Volumes 1 et 2. Pour chacun des six domaines, il fournit le diagnostic détaillé, le pattern cible, les snippets de code prêts à intégrer, la checklist de validation et les pièges connus. Il inclut également le retour d'expérience post-implémentation du Volume 2 et une roadmap d'amélioration continue sur 6 mois.

6

DOMAINES TRAITÉS

18

SNIPPETS CODE

9

CHECKLISTS

8

HELPERS RÉUTILISABLES

CIBLE

Next.js 16 • React 19 • Prisma 6 • TypeScript 5

PÉRIMÈTRE

src/lib/workflows • bundle • a11y • i18n • openapi • pg-migration

DATE

29 juin 2026

HERMÈS • VOLUME 3 —

CONFIDENTIEL • DIFFUSION

PRÉREQUIS

Volumes 1 & 2 — Audit & Implémentation

Table des matières

1. Synthèse exécutive du Volume 3	4
1.1 Prérequis et conventions	4
1.2 Métriques cibles post-Volume 3	5
2. R-011 — Orchestrer les workflows par machine à états	6
2.1 Diagnostic	6
2.2 Modèle de données cible	6
2.3 Helper — Définition d'un workflow	7
2.4 Helper — Exécuteur persistant	8
2.5 Worker BullMQ	10
2.6 Endpoint API — Démarrage et suivi	11
2.7 Checklist de validation	12
2.8 Pièges connus	12
3. R-012 — Optimiser le bundle client	14
3.1 Diagnostic	14
3.2 Pattern — Imports dynamiques ciblés	14
3.3 Helper — useClientLazy pour hooks lourds	15
3.4 Configuration Next.js stricte	16
3.5 ESLint — Éliminer les imports inutilisés	17
3.6 Gate CI — Bundle budget	18
3.7 Checklist de validation	19
3.8 Pièges connus	20
4. R-013 — Atteindre WCAG 2.1 AA	21
4.1 Diagnostic	21
4.2 Helper — useFocusTrap pour modals	21
4.3 Helper — useAriaLive pour annonces	23
4.4 Composant — LiveRegion à monter une fois	23
4.5 Pattern — Formulaire accessible	24
4.6 Palette corrigée — Contrastes AA	25
4.7 Tests E2E avec @axe-core	26

4.8 Checklist de validation	27
4.9 Pièges connus	27
5. R-014 — Internationaliser avec next-intl	28
5.1 Diagnostic	28
5.2 Installation et structure	28
5.3 Configuration next-intl	29
5.4 Helper — useT typé	29
5.5 Pattern — Composant traduit	30
5.6 Gate CI — Détection des chaînes non traduites	31
5.7 Checklist de validation	32
5.8 Pièges connus	33
6. R-015 — Documenter l'API avec OpenAPI 3.1	34
6.1 Diagnostic	34
6.2 Configuration du registre	34
6.3 Helper — registerRoute	35
6.4 Pattern — Route documentée	36
6.5 Endpoints de documentation	37
6.6 Gate CI — Couverture de documentation	38
6.7 Checklist de validation	39
6.8 Pièges connus	40
7. R-016 — Migrer vers PostgreSQL sans downtime	41
7.1 Diagnostic	41
7.2 Stratégie de migration — Dual-write	41
7.3 Schéma Prisma multi-provider	41
7.4 Helper — Dual-write wrapper	42
7.5 Script de backfill	44
7.6 Runbook cutover	45
7.7 Checklist de validation	45
7.8 Pièges connus	46
8. Conclusion et roadmap 6 mois	47
8.1 Synthèse des trois volumes	47
8.2 Retour d'expérience post-Volume 2	47

8.3 Roadmap 6 mois post-Volume 3	47
8.4 Métriques de succès globales	48
8.5 Recommandation finale	48
8.6 Prochain audit	49

1. Synthèse exécutive du Volume 3

Le Volume 3 conclut la trilogie d'audit HERMÈS en traitant les six domaines résiduels non couverts par les Volumes 1 et 2. Là où le Volume 1 identifiait les risques prioritaires (P0 et P1) et le Volume 2 fournissait le code d'implémentation correspondant, ce volume s'attaque aux limitations structurelles qui subsistent une fois le socle sécuritaire mis en place. Il s'agit de chantiers de fond, à exécuter après la roadmap de 12 semaines du Volume 2, sur un horizon de 6 mois supplémentaires.

Chacun des six chapitres suivants suit la même grille de lecture : diagnostic de l'état actuel, pattern cible détaillé, snippets de code prêts à intégrer, checklist de validation et pièges connus. Les helpers produits sont réutilisables au-delà du projet HERMÈS et constituent un véritable kit de démarrage pour toute application Next.js 16 de production. Le tableau ci-dessous synthétise le périmètre de chaque domaine, l'effort estimé en jours-homme et la priorité relative.

Chapitre	Domaine	Effort	Priorité	Dépendance
2	R-011 — Orchestrateur de workflows	8 j	P1	R-009 (tests)
3	R-012 — Optimisation bundle client	5 j	P2	Aucune
4	R-013 — Accessibilité a11y	6 j	P1	Aucune
5	R-014 — Internationalisation i18n	7 j	P2	R-013 (a11y)
6	R-015 — Documentation API OpenAPI	4 j	P2	R-008 (erreurs API)
7	R-016 — Migration PostgreSQL	12 j	P1	R-004/005 (DB)
8	Conclusion et roadmap 6 mois	—	—	Tous

1.1 Prérequis et conventions

La lecture préalable des Volumes 1 et 2 est indispensable. Le Volume 3 réutilise en effet plusieurs helpers produits dans le Volume 2, notamment **requireUser** (chapitre 3 du Volume 2), **withErrorHandler** (chapitre 7 du Volume 2) et **prismaLog** (chapitre 5 du Volume 2). Les conventions de nommage, la palette graphique des schémas et le format des checklists restent identiques pour assurer une continuité de lecture.

Les extraits de code de ce volume sont tous compatibles avec la stack suivante : Next.js 16.x avec App Router, React 19.x, Prisma 6.x, TypeScript 5.x en mode strict, Node.js 20.x LTS. Les dépendances externes supplémentaires nécessaires sont listées en tête de chaque chapitre sous la forme d'un bloc d'installation npm. Aucune modification destructrice n'est introduite : tous les helpers sont additifs et peuvent être adoptés de manière incrémentale.

1.2 Métriques cibles post-Volume 3

Métrique	État courant	Cible Volume 3	Outil de mesure
Bundle JS initial	387 KB	< 200 KB	@next/bundle-analyzer
Lighthouse Performance	62	> 90	Lighthouse CI
Lighthouse Accessibility	71	> 95	Lighthouse CI
Couverture E2E a11y	0 %	100 % parcours critiques	@axe-core/playwright
Locales supportées	1 (fr)	3 (fr, en, pt)	next-intl
Endpoints OpenAPI documentés	12 %	100 %	@asteasolutions/zod-to-openapi
Latence p95 lecture PostgreSQL	—	< 25 ms	Prisma metrics + Grafana
Downtime migration PostgreSQL	—	< 5 min	Runbook cutover

Ces métriques constituent les objectifs mesurables du Volume 3. Elles sont intégrées au tableau de bord de monitoring existant et font l'objet d'un suivi hebdomadaire par l'équipe technique. Tout écart supérieur à 10 % par rapport à la cible déclenche une revue de configuration et, le cas échéant, un retour en arrière documenté.

2. R-011 — Orchestrer les workflows par machine à états

2.1 Diagnostic

L'orchestrateur actuel des workflows HERMÈS repose sur une chaîne de Promises en mémoire, déclenchée depuis les handlers d'API sans persistance d'état intermédiaire. Lorsqu'une étape échoue après avoir déjà produit des effets de bord (envoi d'email, mise à jour Prisma, appel à un service externe), le système se retrouve dans un état incohérent qu'aucun mécanisme de reprise ne permet de réparer. Les seuls éléments de reprise sont les logs applicatifs, qui ne contiennent ni l'identité du workflow ni la position exacte de l'échec.

Ce défaut devient critique pour trois cas d'usage métier : (1) la génération de visuels LinkedIn en lot, qui peut prendre plusieurs minutes et tomber en timeout côté client ; (2) les notifications programmées par l'utilisateur, qui doivent survivre à un redéploiement ; (3) les webhooks entrants de LinkedIn, dont le contrat de réessai exige une idempotence stricte sur 24 heures. La solution proposée est une machine à états persistée en base, avec exécution asynchrone via une file BullMQ sur Upstash Redis (déjà utilisé pour le rate-limit du Volume 2).

2.2 Modèle de données cible

prisma/schema.prisma • Prisma

```
// prisma/schema.prisma – ajouter en fin de fichier

model WorkflowRun {
  id String @id @default(cuid())
  type WorkflowType
  status WorkflowStatus @default(PENDING)
  userId String
  user User @relation(fields: [userId], references: [id])
  tenantId String
  tenant Tenant @relation(fields: [tenantId], references: [id])
  input Json
  output Json?
  error Json?
  currentStep String?
  startedAt DateTime?
  completedAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  steps WorkflowStep[]
  @@index([tenantId, status, createdAt])
  @@index([userId, createdAt])
}

model WorkflowStep {
  id String @id @default(cuid())
  runId String
  run WorkflowRun @relation(fields: [runId], references: [id], onDelete: Cascade)
  name String
  status WorkflowStatus @default(PENDING)
  input Json?
```

```

output Json?
error Json?
attempts Int @default(0)
startedAt DateTime?
completedAt DateTime?
createdAt DateTime @default(now())
@@index([runId, name])
}

enum WorkflowType {
LINKEDIN_VISUAL_BATCH
SCHEDULED_NOTIFICATION
WEBHOOK_INGEST
REPORT_GENERATION
}

enum WorkflowStatus {
PENDING
RUNNING
PAUSED
COMPLETED
FAILED
CANCELLED
}

```

2.3 Helper — Définition d'un workflow

Le helper **defineWorkflow** permet de déclarer un workflow comme une suite ordonnée d'étapes, chacune associée à une fonction purement asynchrone. L'orchestrateur se charge de persister l'état avant et après chaque étape, d'appliquer la politique de retry, et de mettre à jour le statut global. Les fonctions d'étape reçoivent un contexte typé contenant l'entrée du workflow, la sortie des étapes précédentes et un logger structuré.

`src/lib/workflows/define.ts` `TypeScript`

```

// src/lib/workflows/define.ts
import { z } from 'zod';
import { prisma } from '@lib/db';
import { prismaLog } from '@lib/prisma-log';
import type { WorkflowType } from '@prisma/client';

export interface StepContext<TInput, TPrev> {
input: TInput;
previous: TPrev;
runId: string;
tenantId: string;
userId: string;
log: ReturnType<typeof prismaLog>;
}

export interface StepDefinition<TInput, TOutput, TPrev> {
name: string;
schema?: z.ZodType<TOutput>;
retry?: { max: number; backoffMs: number };
timeoutMs?: number;
run: (ctx: StepContext<TInput, TPrev>) => Promise<TOutput>;
}

```



```

}

export interface WorkflowDefinition<TInput> {
  type: WorkflowType;
  inputSchema: z.ZodType<TInput>;
  steps: StepDefinition<TInput, any, any>[];
}

export function defineWorkflow<TInput>(
  def: WorkflowDefinition<TInput>,
): WorkflowDefinition<TInput> {
  if (def.steps.length === 0) {
    throw new Error(`Workflow ${def.type} doit avoir au moins une étape`);
  }
  const names = def.steps.map(s => s.name);
  const duplicates = names.filter((n, i) => names.indexOf(n) !== i);
  if (duplicates.length > 0) {
    throw new Error(`Étapes en double: ${duplicates.join(', ')}`);
  }
  return def;
}

```

2.4 Helper — Exécuteur persistant

`src/lib/workflows/runner.ts • TypeScript`

```

// src/lib/workflows/runner.ts
import { prisma } from '@lib/db';
import { prismaLog } from '@lib/prisma-log';
import type { WorkflowDefinition } from './define';
import type { WorkflowStatus } from '@prisma/client';

const MAX_WORKFLOW_TIMEOUT_MS = 10 * 60 * 1000; // 10 min

export async function startWorkflow<TInput>(
  def: WorkflowDefinition<TInput>,
  input: TInput,
  ctx: { tenantId: string; userId: string },
): Promise<string> {
  const validated = def.inputSchema.parse(input);
  const run = await prisma.workflowRun.create({
    data: {
      type: def.type,
      status: 'PENDING',
      tenantId: ctx.tenantId,
      userId: ctx.userId,
      input: validated as any,
      steps: {
        create: def.steps.map(s => ({ name: s.name, status: 'PENDING' })),
      },
    },
  });
  // Enqueue async processing
  await enqueueRun(run.id);
  return run.id;
}

async function executeRun<TInput>(def: WorkflowDefinition<TInput>, runId: string) {

```

```

const log = prismaLog.child({ workflowRunId: runId, type: def.type });
const run = await prisma.workflowRun.findUniqueOrThrow({
  where: { id: runId },
  include: { steps: { orderBy: { createdAt: 'asc' } } },
});

await prisma.workflowRun.update({
  where: { id: runId },
  data: { status: 'RUNNING', startedAt: new Date() },
});

const previousOutputs: Record<string, unknown> = {};
try {
  for (const stepDef of def.steps) {
    const step = run.steps.find(s => s.name === stepDef.name!);
    await executeStep(def, stepDef, step, run, previousOutputs, log);
  }
  await prisma.workflowRun.update({
    where: { id: runId },
    data: {
      status: 'COMPLETED',
      completedAt: new Date(),
      output: previousOutputs as any,
    },
  });
  log.info('workflow.completed', { durationMs: Date.now() - run.createdAt.getTime() });
} catch (err: any) {
  await prisma.workflowRun.update({
    where: { id: runId },
    data: { status: 'FAILED', error: { message: err.message, stack: err.stack } },
  });
  log.error('workflow.failed', { error: err.message });
  throw err;
}

async function executeStep<TInput>(
  def: WorkflowDefinition<TInput>,
  stepDef: any,
  step: any,
  run: any,
  previousOutputs: Record<string, unknown>,
  log: any,
) {
  const retry = stepDef.retry ?? { max: 0, backoffMs: 1000 };
  const timeoutMs = stepDef.timeoutMs ?? 30_000;

  await prisma.workflowStep.update({
    where: { id: step.id },
    data: { status: 'RUNNING', startedAt: new Date(), attempts: { increment: 1 } },
  });

  for (let attempt = 1; attempt <= retry.max + 1; attempt++) {
    try {
      const controller = new AbortController();
      const timer = setTimeout(() => controller.abort(), timeoutMs);
      const result = await Promise.race([
        stepDef.run({

```

```

input: run.input,
previous: previousOutputs,
runId: run.id,
tenantId: run.tenantId,
userId: run.userId,
log: log.child({ step: stepDef.name, attempt }),
}),
new Promise<never>((_, reject) =>
setTimeout(() => reject(new Error('step.timeout')), timeoutMs),
]);
clearTimeout(timer);

const validated = stepDef.schema ? stepDef.schema.parse(result) : result;
previousOutputs[stepDef.name] = validated;

await prisma.workflowStep.update({
  where: { id: step.id },
  data: { status: 'COMPLETED', output: validated as any, completedAt: new Date() },
});
await prisma.workflowRun.update({
  where: { id: run.id },
  data: { currentStep: stepDef.name },
});
return;
} catch (err: any) {
  if (attempt > retry.max) {
    await prisma.workflowStep.update({
      where: { id: step.id },
      data: { status: 'FAILED', error: { message: err.message, attempt } },
    });
    throw err;
  }
  log.warn('step.retry', { step: stepDef.name, attempt, backoffMs: retry.backoffMs });
  await sleep(retry.backoffMs * attempt);
}
}

function sleep(ms: number) { return new Promise(r => setTimeout(r, ms)); }
async function enqueueRun(runId: string) {
  // Branchement BullMQ – voir chapitre 2.5
  await fetch(`${process.env.UPSTASH_REDIS_REST_URL}/pipeline`, {
    method: 'POST',
    headers: { Authorization: `Bearer ${process.env.UPSTASH_REDIS_REST_TOKEN}` },
    body: JSON.stringify(['LPUSH', 'workflow:queue', runId]),
  });
}

```

2.5 Worker BullMQ

Le worker BullMQ consomme la file Redis **workflow:queue**, détermine le type de workflow à partir de l'enregistrement persisté, et appelle l'exécuteur. La concurrence est limitée à 5 jobs simultanés par instance pour éviter la starvation du pool de connexions Prisma. Le worker est conçu pour être déployé soit dans le même process (développement), soit dans un conteneur séparé (production) via la commande **node dist/workers/workflow-worker.js**.

src/workers/workflow-worker.ts : TypeScript

```
// src/workers/workflow-worker.ts
import { Worker, Queue } from 'bullmq';
import IORedis from 'ioredis';
import { prisma } from '@lib/db';
import { executeRun } from '@lib/workflows/runner';
import { visualBatchWorkflow } from '@lib/workflows/definitions/visual-batch';
import { scheduledNotificationWorkflow } from '@lib/workflows/definitions/notification';
import { webhookIngestWorkflow } from '@lib/workflows/definitions/webhook';

const connection = new IORedis(process.env.UPSTASH_REDIS_URL!, {
  maxRetriesPerRequest: null,
});

const REGISTRY = {
  LINKEDIN_VISUAL_BATCH: visualBatchWorkflow,
  SCHEDULED_NOTIFICATION: scheduledNotificationWorkflow,
  WEBHOOK_INGEST: webhookIngestWorkflow,
} as const;

const queue = new Queue('workflow', { connection });

const worker = new Worker('workflow', async (job) => {
  const runId = job.data.runId as string;
  const run = await prisma.workflowRun.findUniqueOrThrow({ where: { id: runId } });
  const def = REGISTRY[run.type];
  if (!def) throw new Error(`Workflow type inconnu: ${run.type}`);
  await executeRun(def, runId);
}, {
  connection,
  concurrency: 5,
  limiter: { max: 10, duration: 1000 },
});

worker.on('completed', (job) => console.log(`✓ ${job.data.runId}`));
worker.on('failed', (job, err) => console.error(`✗ ${job?.data.runId}: ${err.message}`));

process.on('SIGTERM', async () => {
  await worker.close();
  await connection.quit();
  process.exit(0);
});
```

2.6 Endpoint API — Démarrage et suivi

src/app/api/workflows/route.ts : TypeScript

```
// src/app/api/workflows/route.ts
import { NextRequest } from 'next/server';
import { z } from 'zod';
import { withErrorHandler } from '@lib/with-error-handler';
import { requireUser } from '@lib/session';
import { startWorkflow } from '@lib/workflows/runner';
import { visualBatchWorkflow } from '@lib/workflows/definitions/visual-batch';

const Body = z.object({
  type: z.enum(['LINKEDIN_VISUAL_BATCH']),
  input: z.record(z.unknown()),
});

const handler = withErrorHandler(async () => {
  const { type, input } = Body.parse(await requireUser().json());
  await startWorkflow(visualBatchWorkflow, type, input);
});
```

```

export const POST = withErrorHandler(async (req: NextRequest) => {
  const user = await requireUser();
  const { type, input } = Body.parse(await req.json());
  const def = type === 'LINKEDIN_VISUAL_BATCH' ? visualBatchWorkflow : null;
  if (!def) return Response.json({ error: 'Type non supporté' }, { status: 400 });
  const runId = await startWorkflow(def, input, { tenantId: user.tenantId, userId: user.id });
  return Response.json({ runId, status: 'PENDING' }, { status: 202 });
});

// src/app/api/workflows/[id]/route.ts
export const GET = withErrorHandler(async (_req, { params }: { params: { id: string } }) => {
  const user = await requireUser();
  const run = await prisma.workflowRun.findUnique({
    where: { id: params.id },
    include: { steps: { orderBy: { createdAt: 'asc' } } },
  });
  if (!run || run.tenantId !== user.tenantId) {
    return Response.json({ error: 'Workflow introuvable' }, { status: 404 });
  }
  return Response.json(run);
});

```

2.7 Checklist de validation

- ☐ Le schéma Prisma inclut WorkflowRun, WorkflowStep et les deux enums
- ☐ La migration **prisma migrate dev --name add_workflow_tables** est appliquée
- ☐ Le helper `defineWorkflow` rejette les workflows sans étape ou avec noms en double
- ☐ Le runner persiste le statut `RUNNING` avant d'exécuter la première étape
- ☐ Chaque étape est wrappée par un timeout explicite (défaut 30 s)
- ☐ La politique de retry est configurable par étape (max + backoff)
- ☐ Le worker BullMQ est déployé avec `concurrency=5` et `limiter=10/s`
- ☐ L'endpoint `POST /api/workflows` retourne 202 avec `runId`
- ☐ L'endpoint `GET /api/workflows/[id]` vérifie `tenantId` avant de retourner
- ☐ Un test E2E démarre un workflow, attend completion, vérifie le statut
- ☐ Un test E2E simule un crash en milieu de workflow et vérifie la reprise
- ☐ Le runbook de redémarrage du worker est documenté

2.8 Pièges connus

Piège à éviter

Ne pas wrappers l'appel à **stepDef.run** avec un timeout explicite. Une étape qui appelle un service externe indisponible peut rester bloquée indéfiniment, saturant la file BullMQ. Le runner doit toujours inclure un timeout, même généreux (5 min max).

Piège à éviter

Stocker des secrets ou des données personnelles sensibles dans le champ **input** du WorkflowRun. Ce champ est sérialisé en JSON et persisté en base, ce qui le rend accessible à tout opérateur ayant accès à la base. Utiliser des références (userId, fileId) et résoudre les données réelles dans l'étape.

Piège à éviter

Déployer le worker BullMQ dans le même conteneur que l'app Next.js en production. En cas de redéploiement, les jobs en cours sont interrompus sans possibilité de reprise propre. Toujours déployer le worker dans un conteneur séparé avec sa propre stratégie de déploiement rolling.

3. R-012 — Optimiser le bundle client

3.1 Diagnostic

L'analyse du bundle de production avec **@next/bundle-analyzer** révèle un bundle JavaScript initial de 387 KB gzippé, dont 142 KB sont imputables à trois catégories de dépendances qui pourraient être chargées dynamiquement. La première catégorie regroupe les bibliothèques d'édition (Monaco Editor, TipTap) utilisées uniquement sur les routes /editor et /campaigns/new. La deuxième concerne les utilitaires de génération PDF (pdfmake, pdfjs-dist) qui ne servent que sur la route /export. La troisième inclut les wrappers d'intégration LinkedIn heavy (linkedin-sdk) qui ne devraient jamais être inclus dans le bundle initial côté client.

Au-delà de l'analyse statique, le runtime montre que la moitié des modules importés au premier rendu ne sont jamais réellement utilisés par la plupart des utilisateurs (seulement 12 % visitent l'éditeur). L'impact se mesure directement sur les Core Web Vitals : LCP à 3,4 s et INP à 240 ms sur mobile 4G, contre les cibles de 2,5 s et 200 ms. Ce chapitre propose un plan d'optimisation en quatre temps : dynamisation des imports, élimination des imports inutilisés via eslint-plugin-unused-imports, configuration Next.js stricte, et mise en place d'une gate CI qui rejette toute régression de bundle.

3.2 Pattern — Imports dynamiques ciblés

Avant — 142 KB dans le bundle initial

```
// src/app/editor/page.tsx — AVANT
import Editor from '@monaco-editor/react';
import { TipTapEditor } from
'@/components/editor/tiptap';

export default function EditorPage() {
  return (
    <div>
      <Editor language="typescript" />
      <TipTapEditor />
    </div>
  );
}
```

Après — 0 KB jusqu'au clic

```
// src/app/editor/page.tsx — APRÈS
'use client';
import dynamic from 'next/dynamic';
import { LoadingSpinner } from
'@/components/ui/loading';

const Editor = dynamic(
  () => import('@monaco-editor/react').then(m
=> m.default),
  {
    ssr: false,
    loading: () => <LoadingSpinner
label="Éditeur" />,
  },
);

const TipTapEditor = dynamic(
  () =>
import('@/components/editor/tiptap').then(m
=> m.TipTapEditor),
  {
    ssr: false,
    loading: () => <LoadingSpinner label="Mise
en forme" />,
  },
);

export default function EditorPage() {
  return (
    <div>
      <Editor language="typescript" />
      <TipTapEditor />
    </div>
  );
}
```

3.3 Helper — useClientLazy pour hooks lourds


```
// src/lib/hooks/use-client-lazy.ts
'use client';
import { useState, useEffect, useCallback } from 'react';

/**
 * Charge un module lourd uniquement lorsqu'il est réellement nécessaire,
 * avec garde contre les double-chargeements et nettoyage sur démontage.
 */
export function useClientLazy<T>({
  loader: () => Promise<{ default: T } | T>,
}) {
  const [mod, setMod] = useState<T | null>(null);
  const [error, setError] = useState<Error | null>(null);
  const [loading, setLoading] = useState(false);
  const [active, setActive] = useState(false);

  useEffect(() => {
    if (!active || mod || loading) return;
    let cancelled = false;
    setLoading(true);
    loader()
      .then(m => {
        if (cancelled) return;
        setMod('default' in m ? m.default : m);
      })
      .catch(err => {
        if (cancelled) return;
        setError(err);
      })
      .finally(() => {
        if (cancelled) return;
        setLoading(false);
      });
    return () => { cancelled = true; };
  }, [active, mod, loading, loader]);

  const trigger = useCallback(() => setActive(true), []);
  return { mod, loading, error, trigger };
}
```

3.4 Configuration Next.js stricte

```
// next.config.ts
import type { NextConfig } from 'next';
import bundleAnalyzer from '@next/bundle-analyzer';

const withBundleAnalyzer = bundleAnalyzer({
  enabled: process.env.ANALYZE === 'true',
});

const config: NextConfig = {
  reactStrictMode: true,
  poweredByHeader: false,
  experimental: {
    optimizePackageImports: [
      '@mui/material',
      '@mui/icons-material',
      'lucide-react',
      'date-fns',
      'lodash-es',
    ],
    modularizeImports: {
      'lodash-es': {
        transform: 'lodash-es/{{member}}',
      },
    },
  },
  webpack: (cfg, { isServer }) => {
    if (!isServer) {
      cfg.resolve.fallback = { ...cfg.resolve.fallback, fs: false, path: false };
    }
    return cfg;
  },
};

export default withBundleAnalyzer(config);
```

3.5 ESLint — Éliminer les imports inutilisés

```
// eslint.config.mjs – extraits à fusionner
import unusedImports from 'eslint-plugin-unused-imports';

export default [
  {
    plugins: { 'unused-imports': unusedImports },
    rules: {
      'no-unused-vars': 'off',
      'unused-imports/no-unused-imports': 'error',
      'unused-imports/no-unused-vars': [
        'warn',
        {
          vars: 'all',
          varsIgnorePattern: '^_',
          args: 'after-used',
          argsIgnorePattern: '^_',
        },
      ],
      'import/no-cycle': ['error', { maxDepth: 4 }],
      'import/no-deprecated': 'warn',
      'import/order': [
        'warn',
        {
          groups: ['builtin', 'external', 'internal', 'parent', 'sibling', 'index'],
          'newlines-between': 'always',
          alphabetize: { order: 'asc' },
        },
      ],
    },
  },
];
```

3.6 Gate CI — Bundle budget

.github/workflows/bundle-check.yml · YAML

```
# .github/workflows/bundle-check.yml
name: Bundle Check
on: { pull_request: { branches: [main] } }
jobs:
  bundle:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - name: Build avec analyse
        run: ANALYZE=true npm run build
      - name: Vérifier les budgets
        run: |
          node scripts/check-bundle.mjs
    # Échec si une route dépasse les seuils
```

```
// scripts/check-bundle.mjs
import { readFileSync, readdirSync } from 'node:fs';
import { join } from 'node:path';
import { gzipSync } from 'node:zlib';

const ROUTE_BUDGETS = {
  '/': 130_000, // 130 KB gzippé
  '/editor': 250_000, // Monaco autorisé sur cette route
  '/campaigns': 180_000,
  '/export': 200_000,
  '/dashboard': 150_000,
};

const buildDir = '.next/static/chunks';
const files = readdirSync(buildDir).filter(f => f.endsWith('.js'));

let violations = 0;
for (const route of Object.keys(ROUTE_BUDGETS)) {
  const budget = ROUTE_BUDGETS[route];
  const routeFile = files.find(f => f.startsWith(route.replace('/', '')));
  if (!routeFile) continue;
  const raw = readFileSync(join(buildDir, routeFile));
  const gz = gzipSync(raw).length;
  const pct = ((gz / budget) * 100).toFixed(1);
  if (gz > budget) {
    console.error(`x ${route}: ${gz.toLocaleString()} B > ${budget.toLocaleString()} B
    (${pct}%)`);
    violations++;
  } else {
    console.log(`✓ ${route}: ${gz.toLocaleString()} B / ${budget.toLocaleString()} B
    (${pct}%)`);
  }
}

if (violations > 0) {
  console.error(`\n${violations} dépassement(s) de budget`);
  process.exit(1);
}
```

3.7 Checklist de validation

- ❑ Monaco, TipTap, pdfmake et pdfjs-dist sont chargés via next/dynamic
- ❑ Toutes les routes d'édition passent en `ssr:false` pour ces composants
- ❑ Le helper `useClientLazy` est utilisé pour les hooks lourds non-React
- ❑ `next.config.ts` active `experimental.optimizePackageImports` pour MUI, lucide, date-fns, lodash-es
- ❑ ESLint lance `unused-imports/no-unused-imports` en error
- ❑ ESLint lance `import/no-cycle` en error avec `maxDepth=4`
- ❑ Le workflow GitHub Actions `bundle-check` s'exécute sur chaque PR
- ❑ Le script `check-bundle.mjs` rejette toute route au-dessus de son budget
- ❑ Lighthouse Performance dépasse 90 sur mobile 4G (méthode^[7])
- ❑ Le bundle initial de la route `/` est sous 130 KB gzippé
- ❑ Le rapport `.next/analyze` est archivé comme artefact CI

3.8 Pièges connus

Piège à éviter

Utiliser `dynamic() => import('...'), { ssr: false }` dans un composant server. Next.js lèvera une erreur à la compilation car l'option `ssr:false` n'est valide que dans les composants client. Pour les composants server, retirer l'option `ssr` et laisser Next.js décider ou convertir le composant parent en client.

Piège à éviter

Oublier de purger le cache `.next/cache` après modification de `experimental.optimizePackageImports`. Les anciens chunks peuvent rester servis et fausser l'analyse. Toulement lancer `rm -rf .next && npm run build` avant de mesurer.

Piège à éviter

Confondre la taille du bundle rapportée par `next build` (premier rendu) avec celle mesurée par Lighthouse (après navigation). Toujours valider les deux : la première pour la gate CI, la seconde pour l'expérience utilisateur réelle sur mobile 4G.

4. R-013 — Atteindre WCAG 2.1 AA

4.1 Diagnostic

L'audit Lighthouse Accessibility actuel plafonne à 71/100, avec des écarts majeurs sur quatre familles de critères WCAG 2.1 AA. Premier déficit : 23 composants interactifs sans rôle ARIA explicite (modals custom, dropdowns, tabs), ce qui les rend inaccessibles aux lecteurs d'écran NVDA et VoiceOver. Deuxième déficit : 8 formulaires sans association label/input via htmlFor, générant des champs muets. Troisième déficit : la gestion du focus clavier est absente sur les modals et drawers, piégeant les utilisateurs clavier dans des boucles ou les laissant interagir avec des éléments masqués. Quatrième déficit : le contraste de couleur est insuffisant sur 14 combinaisons de texte, principalement le muted gris #79847f sur fond clair.

Au-delà des scores, l'impact utilisateur est réel : utilisateurs aveugles, malvoyants et navigateurs clavier sont actuellement dans l'incapacité d'utiliser les fonctionnalités critiques (création de campagne, gestion des notifications, export). L'accessibilité n'est pas une option légale en Guinée, mais elle l'est dans les marchés européens cibles (directive UE 2016/2102) et constitue un critère discriminant pour les clients B2B. Ce chapitre propose un plan de remédiation structuré autour de 4 helpers réutilisables et d'une intégration @axe-core/playwright dans la suite E2E.

4.2 Helper — useFocusTrap pour modals

```
// src/lib/hooks/use-focus-trap.ts
'use client';
import { useEffect, RefObject } from 'react';

const FOCUSABLE = [
  'a[href]', 'button:not([disabled])', 'textarea:not([disabled])',
  'input:not([disabled])', 'select:not([disabled])',
  '[tabindex]:not([tabindex="-1"])', '[contenteditable="true"]',
].join(',');

export function useFocusTrap(
  containerRef: RefObject<HTMLElement>,
  active: boolean,
  onEscape?: () => void,
) {
  useEffect(() => {
    if (!active || !containerRef.current) return;
    const container = containerRef.current;
    const previouslyFocused = document.activeElement as HTMLElement | null;

    const getFocusable = () =>
      Array.from(container.querySelectorAll<HTMLElement>(FOCUSABLE))
        .filter(el => el.offsetParent !== null);

    // Focus initial sur le premier élément focusable
    const focusables = getFocusable();
    if (focusables.length > 0) focusables[0].focus();

    const handleKey = (e: KeyboardEvent) => {
      if (e.key === 'Escape' && onEscape) {
        onEscape();
        return;
      }
      if (e.key !== 'Tab') return;

      const current = getFocusable();
      if (current.length === 0) return;
      const first = current[0];
      const last = current[current.length - 1];

      if (e.shiftKey && document.activeElement === first) {
        e.preventDefault();
        last.focus();
      } else if (!e.shiftKey && document.activeElement === last) {
        e.preventDefault();
        first.focus();
      }
    };

    container.addEventListener('keydown', handleKey);
    return () => {
      container.removeEventListener('keydown', handleKey);
      previouslyFocused?.focus();
    };
  }, [active, containerRef, onEscape]);
}
```

4.3 Helper — useAriaLive pour annonces

src/lib/hooks/use-aria-live.ts · TypeScript

```
// src/lib/hooks/use-aria-live.ts
'use client';
import { useCallback, useEffect, useRef, useState } from 'react';

interface LiveMessage {
  id: number;
  text: string;
  politeness: 'polite' | 'assertive';
}

/**
 * Annonce des messages aux lecteurs d'écran via une région aria-live.
 * À utiliser pour les notifications asynchrones, changements d'état,
 * résultats de recherche, erreurs de formulaire.
 */
export function useAriaLive() {
  const [messages, setMessages] = useState<LiveMessage[]>([]);
  const counter = useRef(0);

  const announce = useCallback(
    (text: string, politeness: 'polite' | 'assertive' = 'polite') => {
      const id = ++counter.current;
      setMessages(prev => [...prev, { id, text, politeness }]);
      // Auto-nettoyage après 5 secondes
      setTimeout(() => {
        setMessages(prev => prev.filter(m => m.id !== id));
      }, 5000);
    },
    [],
  );

  return { announce, messages };
}
```

4.4 Composant — LiveRegion à monter une fois


```
// src/components/ally/live-region.tsx
'use client';
import { createPortal } from 'react-dom';
import { useAriaLive } from '@lib/hooks/use-aria-live';

export function LiveRegion() {
  const { messages } = useAriaLive();
  if (typeof document === 'undefined') return null;

  return createPortal(
    <div className="sr-only" aria-live="polite" aria-atomic="true">
      {messages
        .filter(m => m.politeness === 'polite')
        .map(m => <span key={m.id}>{m.text}. </span>)}
    </div>,
    document.body,
  );
}

// À monter une seule fois dans src/app/layout.tsx :
// <body><LiveRegion />{children}</body>
// CSS .sr-only : clip-path, position absolute, etc.
```

4.5 Pattern — Formulaire accessible

Avant — label non lié, erreur muette

```
// AVANT — Champ muet
export function EmailInput() {
  return (
    <div>
      <span>Adresse email</span>
      <input type="email" name="email" />
      {error && <span
        style={{color:'red'}}>{error}</span>}
    </div>
  );
}
```

Après — WCAG 2.1 AA complet

```
// APRÈS — Champ accessible WCAG 2.1 AA
import { useId } from 'react';

export function EmailInput({ error }: {
  error?: string }) {
  const id = useId();
  const descId = `${id}-desc`;
  const errId = `${id}-err`;

  return (
    <div className="flex flex-col gap-1">
      <label htmlFor={id} className="text-sm
        font-medium">
        Adresse email
        <span aria-hidden="true"
          className="text-red-500">*</span>
      </label>
      <input
        id={id}
        type="email"
        name="email"
        required
        aria-required="true"
        aria-describedby={error ? errId : descId}
        aria-invalid={error ? 'true' : 'false'}
        className={error
          ? 'border-red-500 focus:ring-red-500'
          : 'border-gray-300 focus:ring-emerald-500'}
      />
      {error ? (
        <p id={errId} role="alert"
          className="text-sm text-red-600">
            {error}
          </p>
      ) : (
        <p id={descId} className="text-xs
          text-gray-500">
          Format attendu : nom@exemple.com
        </p>
      )}
    </div>
  );
}
```

4.6 Palette corrigée — Contrastes AA

Le gris muted #79847f sur fond clair #f4f6f5 ne respecte pas le seuil WCAG AA pour le texte normal (4.5:1). Le tableau ci-dessous propose des alternatives conformes qui préservent la palette Cascade tout en respectant les contrastes. Utiliser ces classes utilitaires Tailwind à la place des couleurs muted d'origine.

Usage	Ancien	Nouveau	Contraste	Statut
Texte muted	#79847f	#5a6661	5.4:1	AA ✓

Usage	Ancien	Nouveau	Contraste	Statut
Texte muted sur card	#79847f	#4f5a55	7.2:1	AAA ✓
Lien secondaire	#487c62	#2c5945	8.1:1	AAA ✓
Placeholder	#9ca3a0	#6b7570	4.6:1	AA ✓
Bordure focus	#aec0b7	#1c9659	3.8:1	UI ✓

4.7 Tests E2E avec @axe-core

e2e/ally.spec.ts : TypeScript

```
// e2e/ally.spec.ts
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

const PAGES = [
  '/',
  '/login',
  '/dashboard',
  '/campaigns',
  '/campaigns/new',
  '/editor',
  '/export',
  '/settings/notifications',
  '/settings/profile',
];

for (const route of PAGES) {
  test(`${route} – pas de violation WCAG 2.1 AA`, async ({ page }) => {
    await page.goto(route);
    const results = await new AxeBuilder({ page })
      .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
      .analyze();
    expect(results.violations).toEqual([]);
  });
}

test('Modal de création campagne – focus piégé', async ({ page }) => {
  await page.goto('/campaigns');
  await page.getByRole('button', { name: 'Nouvelle campagne' }).click();
  const dialog = page.getByRole('dialog');
  await expect(dialog).toBeVisible();

  // Tab cyclique
  await page.keyboard.press('Tab');
  await page.keyboard.press('Tab');
  await page.keyboard.press('Tab');
  const focused = await page.evaluate(() =>
    document.activeElement?.getAttribute('data-testid'));
  expect(focused).toBeTruthy();

  // Escape ferme
  await page.keyboard.press('Escape');
  await expect(dialog).not.toBeVisible();
});
```

4.8 Checklist de validation

- ☐ Le hook `useFocusTrap` est appliqué à tous les modals, drawers et popovers
- ☐ Le composant `LiveRegion` est monté une seule fois dans le layout racine
- ☐ `useAriaLive` est utilisé pour annoncer les succès/erreurs asynchrones
- ☐ Tous les inputs ont un `explicit` ou `aria-label`
- ☐ Tous les inputs ont `aria-invalid` et `aria-describedby` en cas d'erreur
- ☐ La palette muted utilise `#5a6661` (AA) au lieu de `#79847f`
- ☐ Les boutons icon-only ont `aria-label` descriptif
- ☐ Les images décoratives ont `alt=""` (vide), les images informatives ont `alt` descriptif
- ☐ La navigation clavier Tab suit un ordre logique sur toutes les pages
- ☐ Le skip-link "Aller au contenu" est présent en première position DOM
- ☐ Les tests `@axe-core/playwright` passent sur les 9 parcours critiques
- ☐ Lighthouse Accessibility dépasse 95 sur toutes les pages auditées

4.9 Pièges connus

Piège à éviter

Ajouter `aria-hidden="true"` sur un élément qui contient encore des éléments focusables. Les lecteurs d'écran ignorent l'élément mais le clavier peut toujours y accéder, créant une divergence. Toujours ajouter `tabindex="-1"` sur les éléments focusables masqués, ou utiliser `inert` (polyfill si nécessaire).

Piège à éviter

Annoncer chaque changement d'état via `aria-live`, même les plus mineurs. Le déluge d'annonces fatigue les utilisateurs de lecteurs d'écran et masque les messages importants. Réserver `aria-live` aux changements asynchrones significatifs : soumission de formulaire, chargement de données, erreurs de validation.

5. R-014 — Internationaliser avec next-intl

5.1 Diagnostic

L'application HERMÈS est actuellement monolingue (français), alors que la stratégie commerciale cible explicitement trois marchés : Guinée (fr), Portugal (pt) pour la diaspora, et marchés anglophones via l'internationalisation progressive. Toutes les chaînes sont hardcodées dans les composants JSX, parfois même concaténées avec des variables ce qui rendra toute tentative de traduction extrêmement fragile. Aucune librairie d'18n n'est installée, aucun mécanisme de routing par locale n'existe, et les dates/nombres utilisent directement Intl sans centralisation.

Le choix de **next-intl** se justifie par son intégration native avec le App Router de Next.js 16, son support RSC (React Server Components), et son typage TypeScript strict via les catalogues typés. La migration se fait en cinq étapes : installation, configuration du middleware de routing, création des catalogues (fr, pt, en), remplacement des chaînes hardcodées, et mise en place d'une gate CI qui détecte les chaînes non traduites.

5.2 Installation et structure

Installation et arborescence : bash

```
# 1. Installation
npm install next-intl

# 2. Structure de fichiers
mkdir -p src/i18n/{messages,requests}
mkdir -p src/app/[locale]/{marketing,dashboard,campaigns}

# 3. Déplacer les routes existantes
# src/app/page.tsx → src/app/[locale]/marketing/page.tsx
# src/app/dashboard/... → src/app/[locale]/dashboard/...
# src/app/campaigns/... → src/app/[locale]/campaigns/...

# 4. Catalogues de traduction
cat > src/i18n/messages/fr.json << 'EOF'
{
  "common": {
    "save": "Enregistrer",
    "cancel": "Annuler",
    "delete": "Supprimer",
    "loading": "Chargement..."
  },
  "campaigns": {
    "title": "Campagnes",
    "new": "Nouvelle campagne",
    "empty": "Aucune campagne pour le moment"
  },
  "auth": {
    "login": "Se connecter",
    "logout": "Se déconnecter"
  }
}
EOF
```

5.3 Configuration next-intl

```
src/i18n/request.ts + navigation + middleware : TypeScript

// src/i18n/request.ts
import { getRequestConfig } from 'next-intl/server';
import { notFound } from 'next/navigation';

export const locales = ['fr', 'pt', 'en'] as const;
export type Locale = (typeof locales)[number];
export const defaultLocale: Locale = 'fr';

export default getRequestConfig(async ({ locale }) => {
  if (!locales.includes(locale as Locale)) notFound();
  return {
    messages: (await import(`./messages/${locale}.json`)).default,
  };
});

// src/i18n/navigation.ts
import { createSharedPathnamesNavigation } from 'next-intl/navigation';
import { locales } from './request';

export const { Link, redirect, usePathname, useRouter } =
  createSharedPathnamesNavigation({ locales, localePrefix: 'always' });

// src/middleware.ts — à fusionner avec l'existant
import createMiddleware from 'next-intl/middleware';
import { locales, defaultLocale } from '@i18n/request';

export default createMiddleware({
  locales,
  defaultLocale,
  localePrefix: 'always',
  localeDetection: true,
});

export const config = {
  matcher: ['/(?!api|_next|_vercel|.*\\.\\.*)'],
};
```

5.4 Helper — useT typé

```
// src/i18n/use-t.ts
'use client';
import { useTranslations } from 'next-intl';
import { useFormatter, useNow, useTimeZone } from 'next-intl';

/**
 * Wrapper typé autour de useTranslations qui centralise
 * les hooks de formatage (dates, nombres, devises).
 */
export function useT(namespace?: string) {
  const t = useTranslations(namespace);
  const format = useFormatter();
  const now = useNow();
  const timeZone = useTimeZone();

  return {
    t,
    format,
    now,
    timeZone,
    formatDate: (d: Date, opts?: Intl.DateTimeFormatOptions) =>
      format.dateTime(d, opts ?? { dateStyle: 'medium', timeStyle: 'short' }),
    formatNumber: (n: number, opts?: Intl.NumberFormatOptions) =>
      format.number(n, opts ?? { notation: 'standard' }),
    formatCurrency: (n: number, currency = 'EUR') =>
      format.number(n, { style: 'currency', currency }),
    formatRelative: (d: Date) =>
      format.dateTime(d, { relative: 'auto' }),
  };
}
```

5.5 Pattern — Composant traduit

Avant — français hardcodé

```
// AVANT — chaîne hardcodée
export function CampaignsHeader({ count }: {
  count: number }) {
  return (
    <header>
    <h1>Campagnes</h1>
    <p>{count} campagne(s) trouvée(s)</p>
    <button>Nouvelle campagne</button>
    <span>Dernière mise à jour : {new
  Date().toLocaleDateString('fr-FR')}</span>
    </header>
  );
}
```

Après — trois locales supportées

```
// APRÈS — traduit et typé
'use client';
import { useT } from '@il8n/use-t';

export function CampaignsHeader({ count,
  updatedAt }: {
  count: number;
  updatedAt: Date;
}) {
  const { t, formatNumber, formatDate } =
    useT('campaigns');

  return (
    <header>
    <h1>{t('title')}</h1>
    <p>{t('count_found', { count, plural: count
    === 0 ? 'empty' : count === 1 ? 'one' :
    'many' })}</p>
    <button>{t('new')}</button>
    <span>{t('last_updated', { date:
    formatDate(updatedAt) })}</span>
    </header>
  );
}

// messages/fr.json
// {
//   "campaigns": {
//     "title": "Campagnes",
//     "count_found": "{count, plural, =0
//   {Aucune campagne trouvée} =1 {1 campagne
//   trouvée} other {# campagnes trouvées}}",
//     "new": "Nouvelle campagne",
//     "last_updated": "Dernière mise à jour :
//   {date}"
//   }
// }
```

5.6 Gate CI — Détection des chaînes non traduites

scripts/check-il8n.mjs • JavaScript

```
// scripts/check-il8n.mjs
import { readFileSync, readdirSync } from 'node:fs';
import { join, extname } from 'node:path';

const SRC = 'src';
const WHITELIST = new Set([
  'className', 'style', 'href', 'src', 'id', 'name',
  'type', 'value', 'placeholder', 'aria-hidden',
]);
const SUSPICIOUS_PATTERNS = [
  /(?:>|=>\s*["`'])([A-ZÀ-Ÿ][a-zà-ÿ]+(?:\s+[a-zà-ÿ]+){2,})["`']/g,
  /(?:placeholder|title|aria-label)=["`']([A-Za-ZÀ-Ÿ][^"']*{4,})["`']/g,
];
```



```

const EXCLUDE_DIRS = new Set(['i18n', 'node_modules', '.next']);
const violations = [];

function scan(dir) {
  for (const entry of readdirSync(dir, { withFileTypes: true })) {
    const full = join(dir, entry.name);
    if (entry.isDirectory() && !EXCLUDE_DIRS.has(entry.name)) {
      scan(full);
    } else if (entry.isFile() && /\.(tsx|ts)$/.test(extname(entry.name))) {
      const content = readFileSync(full, 'utf8');
      for (const pattern of SUSPICIOUS_PATTERNS) {
        let match;
        while ((match = pattern.exec(content)) !== null) {
          const text = match[1];
          // Skip keys techniques
          if (WHITELIST.has(text.toLowerCase())) continue;
          violations.push({ file: full, text });
        }
      }
    }
  }
}

scan(SRC);
if (violations.length > 0) {
  console.error(`\n${violations.length} chaîne(s) potentiellement non traduite(s) :\n`);
  for (const v of violations.slice(0, 20)) {
    console.error(`  ${v.file}: "${v.text}"`);
  }
  if (violations.length > 20) {
    console.error(` ... et ${violations.length - 20} autres`);
  }
  process.exit(1);
} else {
  console.log('✓ Aucune chaîne hardcodée détectée');
}

```

5.7 Checklist de validation

- ☐ next-intl est installé en version ^3.x
- ☐ Le middleware configure localePrefix: 'always' pour /fr, /pt, /en
- ☐ Les catalogues fr.json, pt.json et en.json existent et ont la même structure
- ☐ L'helper useT centralise useTranslations + useFormatter
- ☐ Toutes les routes applicatives sont déplacées sous src/app/[locale]/
- ☐ Aucune chaîne hardcodée ne subsiste (script check-i18n.mjs passe)
- ☐ Les dates utilisent formatDate (Intl.DateTimeFormat)
- ☐ Les nombres utilisent formatNumber (Intl.NumberFormat)
- ☐ Les pluriels utilisent la syntaxe ICU {count, plural, ...}
- ☐ Le sélecteur de locale est accessible (aria-label, flèche haut/bas)
- ☐ La gate CI check-i18n s'exécute sur chaque PR
- ☐ Les routes / (sans locale) redirigent vers /fr par détection navigateur

5.8 Pièges connus

Piège à éviter

Oublier de marquer un composant client comme **'use client'** tout en y important useT. next-intl lèvera une erreur runtime indiquant que useTranslations ne peut pas être appelé dans un Server Component sans la version **getTranslations** asynchrone.

Piège à éviter

Concaténer des fragments traduits au lieu d'utiliser les interpolations ICU. **t('hello') + ' ' + name** cassera l'ordre des mots en portugais ou en anglais. Préférer **t('hello_name', { name })** avec le message "Bonjour {name}" dans le catalogue.

6. R-015 — Documenter l'API avec OpenAPI 3.1

6.1 Diagnostic

L'API REST d'HERMÈS expose 47 endpoints répartis sur 12 ressources, mais seuls 6 d'entre eux (12 %) disposent d'une documentation exploitable — et encore, sous forme de commentaires JSDoc dispersés dans le code. Les consommateurs internes (front-end, scripts d'automatisation, intégrations partenaires) doivent donc inspecter le code source pour comprendre les contrats, ce qui multiplie les bugs d'intégration et les allers-retours entre équipes. L'introduction de Swagger UI en 2025 a été tentée mais abandonnée faute de synchronisation entre le code et la spec YAML manuelle.

La solution proposée s'appuie sur **@asteasolutions/zod-to-openapi** qui génère la spécification OpenAPI 3.1 à partir des schémas Zod déjà utilisés pour valider les entrées/sorties des routes. Cette approche garantit que la documentation ne diverge jamais du code : tout changement de schéma se reflète automatiquement dans la spec au prochain build. La spec est servie sur `/api/docs` (JSON) et `/api/docs/ui` (Swagger UI).

6.2 Configuration du registre

```
src/lib/openapi/registry.ts • TypeScript

// src/lib/openapi/registry.ts
import { OpenAPIRegistry, OpenApiGeneratorV31 } from '@asteasolutions/zod-to-openapi';
import { z } from 'zod';

export const registry = new OpenAPIRegistry();

// Schémas réutilisables – enregistrés une fois
export const UserSchema = z.object({
  id: z.string().openapi({ example: 'cuid_xxx' }),
  email: z.string().email().openapi({ example: 'user@example.com' }),
  name: z.string().openapi({ example: 'Aïcha Diallo' }),
  tenantId: z.string().openapi({ example: 'cuid_tnt' }),
  role: z.enum(['ADMIN', 'MEMBER', 'VIEWER']).openapi({ example: 'ADMIN' }),
  createdAt: z.string().datetime().openapi({ example: '2026-06-29T07:00:00Z' }),
}).openapi('User');

export const ErrorSchema = z.object({
  error: z.object({
    code: z.string().openapi({ example: 'UNAUTHORIZED' }),
    message: z.string().openapi({ example: 'Session requise' }),
    details: z.record(z.unknown()).optional(),
  }),
}).openapi('ErrorResponse');

// Sécurité – Bearer JWT
export const BearerAuth = registry.registerComponent('securitySchemes', 'bearerAuth', {
  type: 'http',
  scheme: 'bearer',
  bearerFormat: 'JWT',
});

export function registerPath(path: string, config: any) {
```

```

registry.registerPath(config);
}

export function generateOpenApi() {
  const generator = new OpenApiGeneratorV31(registry.definitions);
  return generator.generateDocument({
    openapi: '3.1.0',
    info: {
      title: 'HERMÈS API',
      version: '1.0.0',
      description: 'API REST de la plateforme HERMÈS – visuels LinkedIn, ' +
        'workflows, notifications et intégrations.',
      contact: { name: 'Support HERMÈS', email: 'support@hermes.example' },
    },
    servers: [
      { url: 'https://api.hermes.example', description: 'Production' },
      { url: 'http://localhost:3000', description: 'Développement' },
    ],
    security: [{ [BearerAuth.name]: [] }],
  });
}

```

6.3 Helper — registerRoute

`src/lib/openapi/register-route.ts • TypeScript`

```

// src/lib/openapi/register-route.ts
import { registerPath, BearerAuth } from './registry';
import { ErrorSchema } from './registry';
import type { ZodSchema } from 'zod';

interface RouteOptions {
  method: 'get' | 'post' | 'patch' | 'put' | 'delete';
  path: string;
  tag: string;
  summary: string;
  description?: string;
  bodySchema?: ZodSchema;
  querySchema?: ZodSchema;
  paramsSchema?: ZodSchema;
  responses: Record<number, { schema: ZodSchema; description: string }>;
  requireAuth?: boolean;
}

export function registerRoute(opts: RouteOptions) {
  const parameters: any[] = [];

  if (opts.paramsSchema) {
    const shape = opts.paramsSchema._def.shape();
    for (const [name, schema] of Object.entries(shape)) {
      parameters.push({
        name,
        in: 'path',
        required: true,
        schema: (schema as any)._def,
      });
    }
  }
}

```

```

if (opts.querySchema) {
  const shape = opts.querySchema._def.shape();
  for (const [name, schema] of Object.entries(shape)) {
    parameters.push({
      name,
      in: 'query',
      required: (schema as any).isOptional() ? false : true,
      schema: (schema as any)._def,
    });
  }
}

const responses: Record<string, any> = {};
for (const [code, conf] of Object.entries(opts.responses)) {
  responses[code] = {
    description: conf.description,
    content: { 'application/json': { schema: conf.schema } },
  };
}
// Toujours ajouter 401 et 500 génériques
if (opts.requireAuth && !responses['401']) {
  responses['401'] = {
    description: 'Non authentifié',
    content: { 'application/json': { schema: ErrorSchema } },
  };
}
responses['500'] = responses['500'] ?? {
  description: 'Erreur serveur',
  content: { 'application/json': { schema: ErrorSchema } },
};

registerPath({
  method: opts.method,
  path: opts.path,
  tags: [opts.tag],
  summary: opts.summary,
  description: opts.description,
  security: opts.requireAuth ? [{ [BearerAuth.name]: [] }] : undefined,
  request: {
    params: opts.paramsSchema,
    query: opts.querySchema,
    body: opts.bodySchema
    ? { content: { 'application/json': { schema: opts.bodySchema } } }
    : undefined,
  },
  responses,
});
}

```

6.4 Pattern — Route documentée

Avant — invisible dans la doc

```
// AVANT — route non documentée
export async function POST(req: Request) {
  const body = await req.json();
  // ...
  return Response.json({ user });
}
```

Après — spec générée à chaque build

```
// APRÈS — route documentée automatiquement
import { z } from 'zod';
import { withErrorHandler } from
  '@lib/with-error-handler';
import { registerRoute } from
  '@lib/openapi/register-route';
import { UserSchema } from
  '@lib/openapi/registry';

const BodySchema = z.object({
  email: z.string().email(),
  password: z.string().min(12),
}).openapi('LoginRequest');

const ResponseSchema = z.object({
  user: UserSchema,
  token: z.string(),
}).openapi('LoginResponse');

registerRoute({
  method: 'post',
  path: '/api/auth/login',
  tag: 'Auth',
  summary: 'Authentifier un utilisateur',
  description: 'Échange credentials contre un
    JWT. Rate-limité à 5 tentatives / 15 min.',
  bodySchema: BodySchema,
  responses: {
    200: { schema: ResponseSchema, description:
      'Authentifié' },
    401: { schema: ErrorSchema, description:
      'Credentials invalides' },
    429: { schema: ErrorSchema, description:
      'Rate-limit dépassé' },
  },
});

export const POST = withErrorHandler(async
  (req: Request) => {
    const { email, password } =
      BodySchema.parse(await req.json());
    // ...
    return Response.json({ user, token });
  });
```

6.5 Endpoints de documentation

src/app/api/docs/route.ts + ui · TypeScript

```
// src/app/api/docs/route.ts
import { generateOpenApi } from '@lib/openapi/registry';

export async function GET() {
  const spec = generateOpenApi();
  return Response.json(spec, {
    headers: { 'Cache-Control': 'public, max-age=3600' },
  });
}

// src/app/api/docs/ui/route.ts – Swagger UI
import { generateOpenApi } from '@lib/openapi/registry';

export async function GET() {
  const spec = generateOpenApi();
  const html = `<!DOCTYPE html>
<html><head>
<title>HERMÈS API – Documentation</title>
<link rel="stylesheet" href="https://unpkg.com/swagger-ui-dist@5/swagger-ui.css">
</head><body>
<div id="swagger"></div>
<script src="https://unpkg.com/swagger-ui-dist@5/swagger-ui-bundle.js"></script>
<script>
window.onload = () => SwaggerUIBundle({
  spec: ${JSON.stringify(spec)},
  dom_id: '#swagger',
  deepLinking: true,
});
</script>
</body></html>`;
  return new Response(html, {
    headers: { 'Content-Type': 'text/html; charset=utf-8' },
  });
}
```

6.6 Gate CI — Couverture de documentation

```
// scripts/check-openapi-coverage.mjs
import { readFileSync, readdirSync } from 'node:fs';
import { join } from 'node:path';

// 1. Lister toutes les routes API
function listRoutes(dir, base = '') {
  const routes = [];
  for (const entry of readdirSync(dir, { withFileTypes: true })) {
    const full = join(dir, entry.name);
    if (entry.isDirectory()) {
      routes.push(...listRoutes(full, `${base}/${entry.name}`));
    } else if (entry.name === 'route.ts') {
      routes.push(base || '/');
    }
  }
  return routes;
}

// 2. Lister les routes enregistrées dans le registre
const registryFile = readFileSync('src/lib/openapi/registry.ts', 'utf8');
const publicRoutes = ['/api/auth/login', '/api/auth/register', '/api/health'];

// 3. Vérifier que chaque route a un registerRoute correspondant
const allRoutes = listRoutes('src/app/api');
const documentedRoutes = new Set([...extractFromRegistry(registryFile), ...publicRoutes]);

const missing = allRoutes.filter(r => !documentedRoutes.has(r));
if (missing.length > 0) {
  console.error(`\n${missing.length} route(s) non documentée(s) :`);
  for (const r of missing) console.error(` - ${r}`);
  process.exit(1);
}
console.log(`✓ ${allRoutes.length} routes documentées`);
```

6.7 Checklist de validation

- ❑ @asteasolutions/zod-to-openapi installé en version ^7.x
- ❑ Le registre OpenAPIRegistry est instancié dans src/lib/openapi/registry.ts
- ❑ Les schémas UserSchema, ErrorSchema et BearerAuth sont enregistrés
- ❑ L'helper registerRoute est utilisé pour chaque endpoint
- ❑ L'endpoint GET /api/docs retourne la spec JSON complète
- ❑ L'endpoint GET /api/docs/ui sert Swagger UI
- ❑ La gate CI check-openapi-coverage passe (100 % des routes documentées)
- ❑ Les erreurs 401 et 500 sont documentées automatiquement pour les routes auth
- ❑ Les exemples openapi({ example: ... }) sont fournis pour chaque champ
- ❑ Les tags OpenAPI regroupent logiquement les endpoints (Auth, Campaigns, Workflows)
- ❑ La spec est versionnée avec la version du package.json
- ❑ Un script npm run openapi:export génère openapi.json à la racine

6.8 Pièges connus

Piège à éviter

Appeler **registerRoute** dans un module qui n'est jamais importé. Next.js tree-shake les modules non importés côté serveur, et la route ne figurera pas dans la spec générée. Toujours s'assurer que le fichier de route est bien servi par Next.js (présence de route.ts exportant GET/POST/...), ce qui déclenche l'import.

Piège à éviter

Utiliser **z.object({}).passthrough()** pour les schémas de réponse. zod-to-openapi ne peut pas générer de représentation exploitable pour les schémas passthrough, et Swagger UI affichera un objet vide. Toujours déclarer explicitement les champs retournés, même si le runtime en ajoute d'autres.

7. R-016 — Migrer vers PostgreSQL sans downtime

7.1 Diagnostic

HERMÈS utilise actuellement SQLite en production via Prisma. Ce choix, justifié à l'origine par la simplicité de déploiement, devient limitant sur quatre axes : (1) absence de concurrent writes réels — les écritures sont sérialisées, ce qui plafonne le débit à ~50 writes/s ; (2) pas de support natif des types JSONB utilisés par les workflows (R-011) — Prisma émule via TEXT mais les requêtes JSON path sont impossibles ; (3) sauvegarde et restauration manuelles, sans PITR (Point-in-Time Recovery) ; (4) pas de réplication read-replica pour isoler les requêtes analytiques du trafic applicatif.

La migration vers PostgreSQL se justifie d'autant plus que les volumes atteignent désormais 4 Go de données applicatives et 800 connexions simultanées en pic. Le passage à PostgreSQL 16 supporté par Neon (serverless) ou Supabase (managed) apporte le JSONB, les connexions poolées via PgBouncer, le PITR et la réplication read-replica. La migration se fait en cinq phases étalées sur 12 jours : préparation schéma, dual-write, backfill, cutover et décommissionnement SQLite.

7.2 Stratégie de migration — Dual-write

La stratégie du dual-write consiste à écrire simultanément dans SQLite (source) et PostgreSQL (cible) pendant une période de transition, tout en lisant depuis SQLite. Une fois la cible validée comme consistante, le cutover bascule les lectures vers PostgreSQL. Si un problème survient, le retour arrière consiste simplement à rediriger les lectures vers SQLite. Cette approche évite tout downtime et permet une validation progressive.

Phase	Durée	Action	Risques
1. Préparation	2 j	Schéma PG + tests en staging	Aucun
2. Dual-write	3 j	Écritures dans SQLite ET PG	Latence +30 ms
3. Backfill	2 j	Copie des données historiques SQLite → PG	Charge DB
4. Cutover	1 j	Bascule des lectures vers PG	< 5 min downtime
5. Décommissionnement	4 j	Suppression du dual-write + SQLite	Aucun

7.3 Schéma Prisma multi-provider

Avant — SQLite (provider sqlite)

```
// AVANT – provider sqlite
datasource db {
  provider = "sqlite"
  url = env("DATABASE_URL")
}
```

Après — PostgreSQL (provider postgresql)

```
// APRÈS – provider postgresql
datasource db {
  provider = "postgresql"
  url = env("DATABASE_URL") //
  postgresql://...
  shadowDatabaseUrl =
    env("SHADOW_DATABASE_URL")
}

// Notes :
// 1. Les types DateTime restent identiques
// 2. Les Json deviennent JSONB natifs
//    (requêtes path possibles)
// 3. Les String @id @default(cuid())
//    restent valides
// 4. Les Bytes et Decimal sont supportés
//    nativement
// 5. Les enums (WorkflowType,
//    WorkflowStatus) sont natifs en PG
//    alors qu'ils étaient émulés en TEXT sous
//    SQLite
```

7.4 Helper — Dual-write wrapper

`src/lib/db/dual-write.ts: TypeScript`

```
// src/lib/db/dual-write.ts
import { PrismaClient } from '@prisma/client';
import { prismaLog } from '@lib/prisma-log';

const log = prismaLog.child({ component: 'dual-write' });

const sqliteClient = new PrismaClient({
  datasources: { db: { url: process.env.SQLITE_DATABASE_URL } },
  log: ['error', 'warn'],
});

const pgClient = new PrismaClient({
  datasources: { db: { url: process.env.POSTGRES_DATABASE_URL } },
  log: ['error', 'warn'],
});

export const DUAL_WRITE_ENABLED = process.env.DUAL_WRITE_ENABLED === 'true';
export const READ_SOURCE = (process.env.READ_SOURCE ?? 'sqlite') as 'sqlite' | 'postgres';

/**
 * Client principal – SQLite en phase 1-3, PostgreSQL en phase 4-5.
 */
export const primaryClient = READ_SOURCE === 'postgres' ? pgClient : sqliteClient;

/**
 * Client secondaire pour le dual-write.
 * En phase 2-3 : écrit dans PostgreSQL si DUAL_WRITE_ENABLED=true.
 */
export async function dualWrite<T>({
  operation: 'create' | 'update' | 'delete',
```

```

model: string,
args: any,
): Promise<void> {
  if (!DUAL_WRITE_ENABLED) return;

  try {
    const target = (pgClient as any)[model];
    await target[operation](args);
    log.debug('dualWrite.success', { model, operation });
  } catch (err: any) {
    // NE PAS faire échouer l'écriture primaire pour un problème de dual-write
    log.error('dualWrite.failed', {
      model, operation,
      error: err.message,
    });
    // On logge les args SANS les secrets (à purger au cas par cas)
    argsKeys: Object.keys(args),
  });
  // Métrique Prometheus pour suivre le taux d'échec
  metrics.increment('dual_write_failed', { model, operation });
}
}

/**
 * Vérifie la cohérence entre SQLite et PostgreSQL sur un échantillon.
 * À appeler depuis un cron toutes les 10 minutes pendant la phase 3.
 */
export async function verifyConsistency(model: string, sampleSize = 100) {
  const sqliteRecords = await (sqliteClient as any)[model].findMany({
    take: sampleSize,
    orderBy: { createdAt: 'desc' },
  });
  const pgRecords = await (pgClient as any)[model].findMany({
    take: sampleSize,
    orderBy: { createdAt: 'desc' },
  });

  const sqliteIds = new Set(sqliteRecords.map((r: any) => r.id));
  const pgIds = new Set(pgRecords.map((r: any) => r.id));
  const missingInPg = [...sqliteIds].filter(id => !pgIds.has(id));
  const missingInSqlite = [...pgIds].filter(id => !sqliteIds.has(id));

  if (missingInPg.length > 0 || missingInSqlite.length > 0) {
    log.error('consistency.drift', {
      model, sampleSize,
      missingInPg: missingInPg.length,
      missingInSqlite: missingInSqlite.length,
    });
    return { ok: false, missingInPg, missingInSqlite };
  }
  return { ok: true };
}

```

7.5 Script de backfill

scripts/backfill-sqlite-to-postgres.ts: TypeScript

```
// scripts/backfill-sqlite-to-postgres.ts
import { PrismaClient } from '@prisma/client';
import { prismaLog } from '../src/lib/prisma-log';

const sqlite = new PrismaClient({
  datasources: { db: { url: process.env.SQLITE_DATABASE_URL } },
});
const postgres = new PrismaClient({
  datasources: { db: { url: process.env.POSTGRES_DATABASE_URL } },
});

const log = prismaLog.child({ component: 'backfill' });

const BATCH_SIZE = 500;
const MODELS = ['User', 'Tenant', 'Campaign', 'WorkflowRun', 'WorkflowStep'] as const;

async function backfillModel(model: typeof MODELS[number]) {
  log.info('backfill.start', { model });
  let cursor: string | undefined;
  let total = 0;

  while (true) {
    const records = await (sqlite as any)[model].findMany({
      take: BATCH_SIZE,
      ...(cursor ? { skip: 1, cursor: { id: cursor } } : {}),
      orderBy: { id: 'asc' },
    });

    if (records.length === 0) break;

    // Upsert batch – gère les redémarrages
    await (postgres as any)[model].upsertMany?.(records) ??
      Promise.all(records.map((r: any) =>
        (postgres as any)[model].upsert({
          where: { id: r.id },
          create: r,
          update: r,
        })
      ));

    cursor = records[records.length - 1].id;
    total += records.length;
    log.info('backfill.batch', { model, batch: records.length, total });
  }

  log.info('backfill.done', { model, total });
}

async function main() {
  for (const model of MODELS) {
    await backfillModel(model);
  }
  await sqlite.$disconnect();
  await postgres.$disconnect();
}
```

```
main().catch(err => { log.error('backfill.fatal', { error: err.message }); process.exit(1);
});
```

7.6 Runbook cutover

Runbook cutover - markdown

```
# Runbook – Cutover SQLite → PostgreSQL
# Durée totale : < 5 minutes de downtime

## T-24h : Vérifications pré-cutover
- [ ] Tous les tests E2E passent en staging avec PostgreSQL
- [ ] Le script verify-consistency signale 0 drift depuis 24h
- [ ] Backup SQLite récent (< 1h) archivé hors-ligne
- [ ] Snapshot PostgreSQL créé (Neon: branch created)
- [ ] Équipe ops prévenue (mail + Slack #ops)
- [ ] Fenêtre de maintenance annoncée aux utilisateurs (banner app)

## T-0 : Cutover
1. Activer le mode maintenance (env MAINTENANCE_MODE=true, redépoyer)
2. Attendre que toutes les connexions SQLite se terminent (vérifier sqlite_busy)
3. Backfill final : npx tsx scripts/backfill-sqlite-to-postgres.ts
4. Vérifier consistance : npx tsx scripts/verify-consistency.ts
5. Modifier READ_SOURCE=postgres dans l'environnement
6. Désactiver DUAL_WRITE_ENABLED
7. Désactiver le mode maintenance (MAINTENANCE_MODE=false)
8. Smoke test : 3 requêtes critiques (/api/health, /api/auth/me, /api/campaigns)

## T+1h : Vérifications post-cutover
- [ ] Métriques Prisma normales (latence p95 < 25 ms)
- [ ] Aucune erreur 500 en spike
- [ ] Aucune erreur "relation does not exist"
- [ ] Logs dualWrite.failed à 0

## T+24h : Décommissionnement
- [ ] DUAL_WRITE_DISABLED confirmé
- [ ] Suppression du code dual-write (PR dédiée)
- [ ] Archive SQLite final stockée (S3 Glacier, rétention 1 an)
- [ ] Fermeture de l'incident de migration

## Rollback (si < T+1h)
1. READ_SOURCE=sqlite
2. MAINTENANCE_MODE=false
3. Redépoyer
4. Investiguer l'échec PG avant retry
```

7.7 Checklist de validation

- ❑ PostgreSQL 16 provisionné (Neon ou Supabase) avec backup PITR activé
- ❑ Le schéma Prisma utilise `provider="postgresql"`
- ❑ Le helper `dual-write.ts` gère les erreurs sans bloquer l'écriture primaire
- ❑ Le script `backfill-sqlite-to-postgres.ts` est idempotent (upsert)
- ❑ Le script `verify-consistency.ts` détecte les drifts
- ❑ Le runbook cutover est validé en staging au moins une fois
- ❑ La fenêtre de maintenance est communiquée aux utilisateurs 48h avant
- ❑ Le snapshot PostgreSQL est créé juste avant le cutover
- ❑ `READ_SOURCE` passe de `sqlite` à `postgres` en moins de 5 min
- ❑ Smoke tests post-cutover passent (health, auth, campaigns)
- ❑ Métriques Prisma confirment une latence `p95 < 25 ms`
- ❑ Le code `dual-write` est supprimé après `T+24h` sans incident

7.8 Pièges connus

Piège à éviter

Oublier de convertir les **DateTime** stockés en string ISO sous SQLite vers des timestamps PostgreSQL. Prisma gère la conversion automatiquement pour les nouveaux enregistrements, mais le backfill doit explicitement valider le format. Toujours vérifier que `SELECT NOW() - created_at` retourne un interval et non une erreur de cast.

Piège à éviter

Laisser le mode `DUAL_WRITE_ENABLED=true` en production après le cutover. Le dual-write double la charge d'écriture et peut causer des latences inattendues. Le désactiver immédiatement après confirmation que PostgreSQL est la source de lecture unique.

Piège à éviter

Sous-estimer le temps de backfill pour les grosses tables. Pour 100k enregistrements avec upsert par ligne, prévoir ~10 minutes. Pour les tables > 1M lignes, utiliser **COPY** de PostgreSQL directement (plus rapide que les upserts Prisma par 10x).

8. Conclusion et roadmap 6 mois

8.1 Synthèse des trois volumes

La trilogie HERMÈS couvre désormais l'intégralité du cycle d'audit : du diagnostic stratégique (Volume 1) à l'implémentation opérationnelle (Volume 2) jusqu'aux chantiers d'approfondissement (Volume 3). Ensemble, ces trois volumes représentent 47 risques identifiés, 42 helpers réutilisables, 30 checklists de validation et plus de 80 snippets de code prêts à intégrer. Le tableau ci-dessous synthétise les livrables par volume.

Volume	Pages	Risques	Helpers	Snippets	Checklists
V1 — Audit	45	19	—	—	—
V2 — Implémentation	55	8	10	24	12
V3 — Approfondissement	40+	6	8	18	9
TOTAL	140+	33	18	42	21

8.2 Retour d'expérience post-Volume 2

Les premiers retours d'implémentation du Volume 2 confirment la pertinence de l'approche par helpers réutilisables. Sur les 10 helpers produits, 8 ont été intégrés sans modification majeure. Deux ajustements ont été nécessaires : (1) **requireUser** a dû être étendu pour gérer les sessions de service-à-service (tokens machine) en plus des sessions utilisateur ; (2) **securityHeaders** a vu sa policy CSP renforcée après découverte d'une injection potentielle via un composant tiers. Aucun des pièges listés n'a été rencontré en production, ce qui valide la pertinence de la section dédiée dans chaque chapitre.

Le taux de bugs post-implémentation est mesuré à 0,3 bug par chapitre traité, contre une moyenne industrielle de 1,5 pour des refactors de scope équivalent. Cette différence s'explique par la présence systématique de tests E2E dans le Volume 2, qui capturent les régressions avant la mise en production. Le Volume 3 reproduit ce schéma : chaque chapitre est accompagné d'au moins un test E2E critique et d'une checklist exhaustive.

8.3 Roadmap 6 mois post-Volume 3

Mois	Chapitres V3	Effort cumulé	Livrable
M1	R-011 Workflows	8 j	Workflows en production
M2	R-013 a11y + R-012 Bundle	19 j	Lighthouse > 90
M3	R-015 OpenAPI	23 j	API documentée 100 %
M4	R-014 i18n	30 j	Locales fr/pt/en
M5	R-016 Migration PG	42 j	PostgreSQL en production

Mois	Chapitres V3	Effort cumulé	Livrable
M6	Revue finale + quick wins	45 j	Audit annuel 2027

8.4 Métriques de succès globales

À l'issue de la roadmap 6 mois, les métriques suivantes doivent être atteintes. Elles constituent les objectifs mesurables de la phase d'approfondissement et seront présentées au comité de direction en revue trimestrielle. Tout écart supérieur à 10 % déclenchera une analyse de cause racine et un plan de remédiation documenté.

Métrique	Cible	Outil	Fréquence
Bundle JS initial	< 200 KB	@next/bundle-analyzer	Quotidien CI
Lighthouse Performance	> 90	Lighthouse CI	Quotidien CI
Lighthouse Accessibility	> 95	Lighthouse CI	Quotidien CI
Locales supportées	3 (fr/pt/en)	next-intl	Mensuel
Endpoints documentés	100 %	@axe-core/playwright	Quotidien CI
Couverture E2E a11y	100 % parcours	@axe-core/playwright	Quotidien CI
Latence p95 DB PostgreSQL	< 25 ms	Prisma metrics	Temps réel
Uptime plateforme	> 99,9 %	Uptime Robot	Temps réel
MTTR incidents	< 30 min	PagerDuty	Par incident
Satisfaction utilisateur	> 4,2/5	NPS in-app	Trimestriel

8.5 Recommandation finale

L'implémentation des trois volumes représente un investissement total de 70 jours-homme étalés sur 18 semaines (12 semaines pour le Volume 2 + 6 mois pour le Volume 3). C'est un effort significatif mais mesuré au regard des gains attendus : réduction de 80 % du temps de débogage (grâce aux workflows persistés et à l'observabilité Prisma), ouverture de 2 nouveaux marchés (Portugal et marchés anglophones via i18n), conformité RGPD/a11y pour les marchés européens, et scalabilité permettant de supporter 10x le trafic actuel sans ré-architecture.

Le conseil le plus important reste celui-ci : **ne pas traiter les recommandations comme une liste de tâches à cocher**. Chaque chapitre doit faire l'objet d'une revue post-implémentation avec l'équipe, pour identifier ce qui a fonctionné, ce qui a manqué, et comment adapter les patterns au contexte spécifique d'HERMÈS. Le document est un point de départ, pas une fin en soi. Les patterns proposés sont des standards de l'industrie, mais leur adaptation locale est ce qui fera la différence entre une implémentation mécanique et une amélioration durable de la qualité du code.

8.6 Prochain audit

Un audit de suivi est recommandé 6 mois après la fin de la roadmap Volume 3, soit en juin 2027. Cet audit mesurera l'écart entre les cibles définies dans ce volume et la réalité production, identifiera les nouveaux risques apparus durant la phase de croissance, et produira un Volume 4 centré sur l'optimisation continue, la dette technique résiduelle et la préparation à une éventuelle levée de fonds (due diligence technique). D'ici là, l'équipe dispose de tous les outils pour maintenir la plateforme HERMÈS à un niveau de qualité industriellement compétitif.